ECE551: Digital System Design and Synthesis

Fall 2026

Testbench Constructs

Department of Electrical and Computer Engineering
UNIVERSITY OF WISCONSIN–MADISON

File I/O

- We may not want to hard-code all information in the testbench
  - Files hold the data, and the testbench provides the structure
- Help automate testing
  - One file with the input stimulus
  - One file with the expected outputs
  - The testbench compares the DUT's outputs against the expected values
- Can have software (Python, MatLab, System C) generate data
  - Create the inputs for testing
  - Create “correct” output values for testing
  - Can use files to “connect” hardware/software system
- Can write results out too: logs, summaries, data for post-processing
- Initialize memories (ROM, flash) from files

File I/O

- Opening / Closing files:
  - **`$fopen`** opens a file and returns an integer descriptor

```
integer fd = $fopen("filename");
integer fd = $fopen("filename", "r");
```

    - If file cannot be open, returns a 0
    - Can output to more than one file simultaneously by writing to the OR (|) of the relevant file descriptors
      - Easier to have “summary” and “detailed” results
  - **`$fclose`** closes the file

```
$fclose(fd);
```

- Writing to files:
  - Take the file descriptor as the first argument, then a formatted string
    - **`$fmonitor()`**
    - **`$fdisplay()`**
    - **`$fstrobe()`**
    - **`$fwrite()`**

```
$fdisplay(fd, "out=%b in=%b", out, in);
```

File I/O

- Reading from files:
  - **`$fgetc()`** – read characters

When finished, the array **`mem`** will contain the characters of this file one by one, and the file will have been echoed to the screen.

```
module file_read();
  parameter EOF = -1;
  integer file_handle,error,indx;
  reg signed [15:0] wide_char;
  reg [7:0] mem[0:255];
  reg [639:0] err_str;

  initial begin
    indx=0;
    file_handle = $fopen("text.txt","r");
    error = $ferror(file_handle,err_str);
    if (error==0) begin
      wide_char = $fgetc(file_handle);
      while (wide_char!=EOF) begin
        mem[indx] = wide_char[7:0];
        $write("%c",mem[indx]);
        indx = indx + 1;
        wide_char = $fgetc(file_handle);
      end
    end
    else $display("Can't open file...");
    $fclose(file_handle);
  end
endmodule
```

The quick brown fox jumped over the lazy dogs

text.txt

File I/O

- Reading from files:
  - **`$fgetc()`** – read characters
  - **`$fgets()`** – read lines

$fgets returns the number of characters it read. 0 means end of file (EOF).

```
module file_read2();
  integer file_handle,error,indx,num_chars_in_line;
  reg [256*8:1] mem[0:255],line_buffer;
  reg [639:0] err_str;

  initial begin
    indx=0;
    file_handle = $fopen("text2.txt","r");
    error = $ferror(file_handle,err_str);
    if (error==0) begin
      num_chars_in_line = $fgets(line_buffer,file_handle);
      while (num_chars_in_line>0) begin
        mem[indx] = line_buffer;
        $write("%s",mem[indx]);
        indx = indx + 1;
        num_chars_in_line = $fgets(line_buffer,file_handle);
      end
      $fclose(file_handle);
    end
    else $display("Could not open file text2.txt");
end
endmodule
```

File I/O

- Reading from files:
  - **`$fgetc()`** – read characters
  - **`$fgets()`** – read lines
  - **`$fscanf()`** – read formatted data

$fscanf returns the number of items matched in a line.

```
module file_read3();
  integer fd,error,idx,num_matches;
  reg [15:0] mem[0:255][0:1];
  reg [639:0] err_str;

  initial begin
    indx=0;
    fd = $fopen("text3.txt","r");
    error = $ferror(fd,err_str);
    if (error==0) begin
      num_matches = $fscanf(fd,"%h %h",mem[idx][0],mem[idx][1]);
      while (num_matches>0) begin
        $display("data is: %h %h",mem[idx][0],mem[idx][1]);
        idx = idx + 1;
        num_matches = $fscanf(fd,"%h %h",mem[idx][0],mem[idx][1]);
      end
      $fclose(fd);
    end
    else $display("Could not open file text3.txt");
  end
endmodule
```

12f3	13f3
abcd	1234
3214	21ab

text3.txt

File I/O

- Loading memory data from files:
- **$readmemh** reads data as hex.
- **$readmemb** reads data as binary.

```
$readmemh("<file_name>",<memory>);
$readmemh("<file_name>",<memory>,<start_addr>,<finish_addr>);
```

They are reading ASCII files either way. The only difference is how the numbers are interpreted.

```
// addr    data
@0000       A2
@0001       B9
@0002       23
```

example "hex" file

```
// addr      data
@0000     10100010
@0001     10111001
@0040     00100011
```

example "binary" file

```
//data
A2
B9
23
```

address is optional for the lazy

```
module rom(input clk, input [7:0] addr, output [15:0] dout);
  reg [15:0] mem[0:255];  // 16-bit wide 256 entry ROM
  reg [15:0] dout;

  initial
    $readmemh("constants",mem);

  always @(negedge clk) begin
    ////////////////////////////////////
    // ROM presents data on clock low //
    ////////////////////////////////////
    dout <= mem[addr];
  end
endmodule
```

Functions

- Declared and referenced within a module
  - Called from expressions (RHS)
  - Can call other functions
- Synthesizable functions implement combinational logic
  - No timing controls (#, @, wait) and no task calls
  - Must execute in zero simulation time
  - Use blocking assignments
- Arguments and return value
  - Returns value is assigned to the function name (unless a void function)
  - Must have at least one input (optional in SystemVerilog)
  - Can have output arguments (new with System Verilog)
- Uses
  - DUT code: conversions and calculations (e.g., saturation)
  - Testbench: self-checking routines that return pass/fail (Tasks are often a better fit)

Function Example

- Does this synthesize?

```
module word_aligner (word_out, word_in);
  output [7:0] word_out;
  input  [7:0] word_in;
  assign word_out = aligned_word(word_in);    // invoke function

  function [7:0] aligned_word;                // function declaration
    input [7:0] word;
    begin
      aligned_word = word;
      if (aligned_word != 0)
        while (aligned_word[7] == 0) aligned_word = aligned_word << 1;
    end
  endfunction
endmodule
```

size of return value

input to function

return value assigned to function name

Function Example

- Multiple input arguments

```
module arithmetic_unit (result_1, result_2, operand_1, operand_2);
  output [4:0] result_1;
  output [3:0] result_2;
  input  [3:0] operand_1, operand_2;
  assign result_1 = sum_of_operands (operand_1, operand_2);
  assign result_2 = larger_operand (operand_1, operand_2);

  function [4:0] sum_of_operands(input [3:0] operand_1, operand_2);
    sum_of_operands = operand_1 + operand_2;
  endfunction

  function [3:0] larger_operand(input [3:0] operand_1, operand_2);
    larger_operand = (operand_1 >= operand_2) ? operand_1 : operand_2;
  endfunction
endmodule
```

function call

return value

function inputs

Department of Electrical and Computer Engineering
UNIVERSITY OF WISCONSIN–MADISON

Function Example

```
module ALU(RF,mem,instr,cin,dst,src0sel,src1sel);
  input [15:0] RF,mem,instr;
  input cin,src0sel,src1sel;
  output [15:0] dst;

  wire [15:0] src0,src1;

  //////// muxing for Src0 & Src1 busses ////
  assign src0 = src0sel ? RF : ceiling9(mem);
  assign src1 = src1sel ? instr : RF;

  //////// implement adder /////////
  assign dst = src1 + src0 + cin;

  function [15:0] ceiling9(input [15:0] in_word);
    ceiling9 = (in_word[15] && ~&in_word[14:8]) ? 16'hFF00 :
               (~in_word[15] && |in_word[14:8]) ? 16'h00FF :
                in_word;
  endfunction
endmodule
```

function call

Function performs a saturation, limiting the output to a signed number in range: [-256,255]

Synthesizes no problem

Function Example

- Void function with outputs

void function (does not return via the function's name)

```
function void add(input [31:0] a_op, b_op, output [31:0] sum, output co);
  {co,sum} = a_op + b_op;
endfunction
```

function has two outputs in argument list

```
always_comb
  add(.sum(result), .co(carry), .a_op(in1), .b_op(in2));
```

Arguments can be passed by name instead of by position.

Re-Entrant (recursive) Functions: **automatic**

- Use keyword **automatic** to enable stack saving of function working space and enable recursive functions.

```
module top;
  //Define the factorial function
  function automatic integer factorial;
  input [31:0] oper;
  begin
    if (oper>=2)
      factorial = factorial(oper-1)*oper;
    else
      factorial = 1;
  end
  endfunction

  initial begin
    result = factorial(4);
    $display("Result is %d",result);
  end
endmodule
```

Better motivation for **automatic**:

To pass a signal as type **ref** (pointer), the **task/function** has to be declared as **automatic**.

For a **task/function** to be in a shared **package**, it must be declared as **automatic**.

Tasks

- More general than functions

Functions:	**Tasks:**
Can call other functions, but not tasks	Can call other tasks and functions
Zero simulation time: no #, @, or wait	May consume time: #, @, wait allowed
Return a value (or void); used in expressions	No return value; called as a statement
At least one input (Verilog); outputs added in SystemVerilog	Zero or more input, output, or inout arguments

Tasks can modify & monitor global signals too. Not the best style, but can be quite convenient.

Why use tasks?

- Tasks provide the ability to
  - Execute common procedures from multiple places
  - Divide large procedures into smaller ones
  - Declare local variables
- Very handy for testbenches
  - Break common testing routines into tasks
    - Initialization tasks
    - Stimulus generation tasks
    - Self Checking tasks
  - Top level test then becomes mainly calls to tasks.

Task Example

```
task SendCmd(input [7:0] cmd2send, input [15:0] data2send);
  begin
    host_cmd = cmd2send;
    host_data = data2send;
    @(posedge clk);
    send2host = 1'b1;
    @(posedge clk);
    send2host = 1'b0;
  end
endtask
```

Setting global signals

Give a meaningful cmd name using localparam

```
SendCmd(8'h28,16'hABCD);
```

Calling by position

```
SendCmd(.cmd2send(8'h28),.data2send(16'hABCD));
```

Calling by name

Task Example

- Want a generic *wait for signal rise* task that can be passed the signal to monitor, as well as the length of the timeout period

```
task wait4sig(input sig, input int clks2wait);
  fork
    begin: timeout
      repeat(clks2wait) @(posedge clk);
      $display("ERR: timed out waiting for sig in wait4sig");
      $stop();
    end
    begin
      @(posedge sig);  // signal of interest asserted
      disable timeout;
    end
  join
endtask
```

This does not work.
When this task is called it is passed the value of **sig** at the time it was called, so it really isn't monitoring **sig.**

Task Example

- Have to have the signal type be **ref**, not input.
  - This is passing a pointer to the signal, so the task is looking at the current value of signal, not just what it was at time of task invocation.

```
task automatic wait4sig(ref sig, input int clks2wait);
    fork
        begin: timeout
            repeat(clks2wait) @(posedge clk);
            $display("ERR: timed out waiting for sig in wait4sig");
            $stop();
        end
        begin
            @(posedge sig);     // signal of interest asserted
            disable timeout;
        end
    join
endtask
```

If a task uses a **ref** signal type, then it has to be specified as **automatic**.

`include Compiler Directive

- `include *filename*
  - Inserts entire contents of another file at compilation
  - `include localparams near beginning
  - `include tasks/functions near the end if you want them to reference global signals

```
module quadcopter_tb();
  `include "tb_params.sv"  // include params first so defined

  ...

  ...

  `include "tb_tasks.sv"  // after signals are declared
endmodule
```

NOTE: included files have to be located either in the ModelSim project directory, or on the include path

Use of `**include** and tasks in testbenches

```
module cbc_dig_tb();
  `include "tb_params.sv";

  reg clk,rst_n;
  reg [23:0] cmd_snd;      // holds cmd Host is sending to DUT
  reg send_cmd;

  ////// instantiate DUT //////
  DSO_dig iDUT(.clk(clk), .rst_n(rst_n), … .TX(TX), .RX(RX));

  initial begin
    initialize;
    SendCmd({CFG_GAIN,16'h1800});  // set gain to 6b for CH1
    ChkResp(POS_ACK);              // wait for & check response

    PollCaptureDone;               // reads trig status reg till capture d
    SendCmd({DUMP_CH,16'h0000});   // dump capture for CH1
    ...
  end
  ...

  `include "tb_tasks.sv"    // include tb tasks

endmodule
```

tb_params.sv

```
localparam DUMP_CH = 8'h01
localparam CFG_GAIN = 8'h02;
```

tb_tasks.v

```
task initialize;
  ...
endtask

task SendCmd(input [23:0] mstr_cmd);
  ...
endtask

...
```

Packages in System Verilog

- Old school Verilog did not have a shared declaration space. Each **module** had to contain all declarations used within that module, including **tasks**, **functions**, **localparams**, and **typedefs**.
- System Verilog added user defined **packages** to provide a declaration space that can be shared by any module.

- Example package:

```
package alu_types;
  localparam delay = 1;

  typedef enum logic [3:0] {ADD, AND, XOR, SHFT_L, SHFT_R} op_t;
endpackage
```

Packages in System Verilog

```
module simp_alu(src1,src2,cin,op,dst,cout);

typedef enum logic [2:0] {ADD, AND, XOR, SHFT_L, SHFT_R} op_t

input [15:0] src1,src2;      // input busses
input cin;                   // carry in
input [2:0] op;              // operation select

output [15:0] dst;           // result of ALU
output cout;                 // carry output

op_t op_i;

assign op_i = op_t'(op);

// an ALU capable of arithmetic,logical, shift, and zero
assign {cout,dst} = (op_i==ADD)     ?   src1+src2+cin :
                    (op_i==AND)     ?   {1'b0,src1 & src2} :
                    (op_i==XOR)     ?   {1'b0,src1 ^ src2} :
                    (op_i==SHFT_L)  ?   {src1,cin} :
                    (op_i==SHFT_R)  ?   {src1[0],src1[15],src
                    17'h00000;

endmodule
```

- Remember this rather clumsy example use of **`typedef`** from before?
- We had to create an internal version (**`op_i`**) of the opcode and then type cast the 3-bit port variable to be of **`op_t`**.
- This was because the port could not be of type **`op_t`** because it could not be defined outside the scope of the module itself
- **Packages let us do that.**

Packages in System Verilog

- With use of package the port **op** can be defined as **op_t** thus no need for the intermediate signal and cast.

```
package alu_types;
  localparam delay = 1;

  typedef enum logic [2:0] {ADD, AND, XOR, SHFT_L, SHFT_R} op_t;
endpackage

module simp_alu
  import alu_types::*;      // import all defined things in alu_types package
  (input [15:0] src1,src2,
   input cin,
   input op_t op,           // referencing imported type op_t
   output [15:0] dst,
   output cout);

   // an ALU capable of arithmetic, logical, shift, and zero //
   assign {cout,dst} = (op==ADD)    ? src1+src2+cin :
                       (op==AND)    ? {1'b0,src1&src2} :
                       (op==XOR)    ? {1'b0,src1^src2} :
                       (op==SHFT_L) ? {src1,cin} :
                       (op==SHFT_R) ? {src1[0],src1[15],src1} :
                      17'h00000;

endmodule
```

Use of packages for testbench tasks

```
package tb_tasks;

    localparam SET_PTCH  = 8'h02;
    localparam SET_ROLL  = 8'h03;
    localparam SET_YAW   = 8'h04;
    localparam SET_THRST = 8'h05;
    localparam CAL_COPTER = 8'h06;

    localparam POS_ACK = 8'hA5;
    localparam DATA    = 8'hxx;
    localparam CAL_SPEED = 11'h290;      // speed to ru

    task automatic Initialize(ref clk, RST_n, send_cmd
      begin
        send_cmd = 0;
        clk = 0;
        RST_n = 0;                          // assert rese
        repeat (2) @(posedge clk);
        @(negedge clk);                  // on negedge REF_
        RST_n = 1;                          // deasert res
        repeat (500) @(negedge clk);
      end
    endtask

    task automatic SendCmd(input [7:0] cmd_to_send, in
                           ref [7:0] host_cmd, ref [15
```

- Can put localparams and tasks/functions together in same file.
- All tasks/functions declared in a package have to be **`automatic`** if any of their arguments are **`ref`**
- Unfortunately, you cannot reference global signals from the testbench level of hierarchy, so all signals to monitor/control should be passed as type **`ref`**

Use of packages for testbench tasks

```
module QuadCopter_tb();

  import tb_tasks::*;       // import all tasks & functions declared in tb

  //// Interconnects to DUT/support defined as type wire /////
  wire SS_n,SCLK,MOSI,MISO,INT;
  wire RX,TX;
  logic [7:0] resp;             // response from DUT
  logic cmd_sent,resp_rdy;
  wire frnt_ESC, back_ESC, left_ESC, rght_ESC;

  ////// Stimulus is declared as type reg ///////
  reg clk, RST_n;
  reg [7:0] host_cmd;               // command host is sending to DUT
  reg [15:0] data;                  // data associated with command


  initial begin
    Initialize(clk,RST_n,send_cmd);
    ////// Start issuing commands to DUT //////

    SendCmd(CAL_COPTER,16'h0000,host_cmd,data,clk,send_cmd,cmd_sent);

    //// Wait for fall of all ESC signals before checking any speed values
    wait4SigFall(iQuad.any_high_ff2,clk,1000000);
    $display("GOOD: ESC's operating");

    if ((iQuad.spd_frnt>(CAL_SPEED+11'h010)) || (iQuad.spd_frnt<(CAL_SPEED
```

- Any signal a task monitors has to be passed as type **`ref`**, and therefore cannot be declared as **`wire`** (use **`logic`**)
- Calls to tasks will have to pass references to any signals the task drives or monitors. These tasks cannot "monitor" global testbench signals.